

Consegna Settimana 2 / Lezione 5 - Progetto

Relazione

Traccia:

Per agire come un Hacker bisogna capire come pensare fuori dagli schemi. L'esercizio di oggi ha lo scopo di allenare l'osservazione critica.

Dato il codice si richiede allo studente di:

1. Capire cosa fa il programma senza eseguirlo.
2. Individuare nel codice sorgente le casistiche non standard che il programma non gestisce (esempio, comportamenti potenziali che non sono stati contemplati).
3. Individuare eventuali errori di sintassi / logici.
4. Proporre una soluzione per ognuno di essi.

Consegnate un pdf con la lista degli errori (aiutatevi con i numeri di riga per indicarli) descrivendo il perchè è un errore e cosa va fatto per correggerlo

Codice da analizzare:

```
1  import datetime
2
3  while True
4      comando_utente = input("Cosa vuoi sapere? ")
5      if comando_utente == "esci":
6          print("Arrivederci!")
7          break
8      else:
9          print(assistente_virtuale(comando_utente))
10
11 def assistente_virtuale(comando):
12     if comando == "Qual è la data di oggi?":
13         oggi = datetime.datetime.today()
14         risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
15     elif comando == "Che ore sono?":
16         ora_attuale = datetime.datetime.now().time()
17         risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
18     elif comando == "Come ti chiami?":
19         risposta = "Mi chiamo Assistente Virtuale"
20     else:
21         risposta = "Non ho capito la tua domanda."
22     return risposta
23
```

Il punto (1) di questa esercitazione chiede di capire cosa fa questo codice senza eseguirlo. Si può subito notare che nella riga 1 viene importata la libreria *datetime* che serve a gestire date e orari.

Continuando la lettura capiamo che si tratta di un semplice assistente virtuale. Dal codice vediamo subito che ci viene chiesto un input e, più in basso, vediamo che gli input validi riguardano sia la data che l'orario ma anche che possiamo chiedere il nome di questo assistente virtuale.

In conclusione si può chiedere:

1. la data al momento della richiesta, la risposta avrà il formato dd/mm/yyyy;
2. l'orario al momento della richiesta, verranno indicati ora e minuti;
3. il nome di questo assistente.

In più, se si scrive qualcosa di non previsto, l'assistente dà un messaggio di errore e rimanda l'input iniziale e scrivendo *esci* in input, l'assistente saluta l'utente e chiude il programma.

Il punto (2) chiede di individuare casistiche non standard che il programma non sa gestire. Partendo dalle funzioni principali, questo programma non può indicare la data di oggi in un formato diverso da quello della convenzione standard italiana; quando viene richiesto l'orario non vengono indicati i secondi; non riesce a fornire data e orario contemporaneamente ma sono una delle due alla volta; ed infine non può utilizzare fusi orari diversi da quello dell'utente.

Continuando ad analizzare il programma si può vedere che se la sintassi della richiesta non viene rispettata perfettamente, questo assistente non riesce a rispondere alla domanda, riportando il messaggio di errore "Non ho capito la tua domanda.", riproponendo così l'input iniziale.

I punti (3) e (4) richiedono di individuare gli errori nel codice del programma e trovare le relative soluzioni.

Gli errori trovati sono quattro:

1. Il primo errore trovato è quello più evidente.

La funzione *assistente_virtuale* (che va da riga 11 a riga 22) viene definita dopo essere stata utilizzata.

In Python però questo non si può fare. La funzione va definita prima per essere poi chiamata ed utilizzata dopo.

La soluzione è semplice. Bisogna spostare tutto il blocco della funzione prima del ciclo *while*.

2. Continuando l'analisi del codice ci accorgiamo che, nella riga 3, dopo il *while True* manca la punteggiatura.

La soluzione è aggiungere i " : " scrivendo quindi: " *while True:* "

3. Rimanendo all'interno del ciclo while, nella riga 7, all'interno del if, vediamo che il *break* che chiude il programma in caso di input "esci" non è indentato correttamente, si trova fuori dall' if.

Questo comporta che, se l'utente inserisce l'input *esci*, allora il programma scrive a schermo "Arrivederci!", poi arriva al *break* e si chiude. Ma se l'utente scrive un input diverso, essendo il *break* fuori dall'if, il programma non esegue l'if ma arriva comunque al *break*, chiudendosi e saltando l'else.

La soluzione è molto semplice, basta indentare correttamente il *break*, inserendolo dentro l'if. In modo che il programma si chiuda solo quando desiderato.

4. Infine, l'ultimo errore trovato, è nella riga 13.
All'interno della libreria *datetime*, "*datetime.datetime.today()*" non esiste.

La soluzione è scrivere questa parte di codice nella sua forma corretta, ovvero *datetime.date.today()*.

Di sotto lo screen del codice riscritto correttamente e funzionante:

```
1  import datetime
2
3  def assistente_virtuale(comando):
4      if comando == "Quale è la data di oggi?":
5          oggi = datetime.date.today()
6          risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
7      elif comando == "Che ore sono?":
8          ora_attuale = datetime.datetime.now().time()
9          risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
10     elif comando == "Come ti chiami?":
11         risposta = "Mi chiamo Assistente Virtuale"
12     else:
13         risposta = "Non ho capito la tua domanda."
14     return risposta
15
16 while True:
17     comando_utente = input("Cosa vuoi sapere? ")
18     if comando_utente == "esci":
19         print("Arrivederci!")
20         break
21     else:
22         print(assistente_virtuale(comando_utente))
23
```